

Reviewer Pack — Coxeter Group Action Index and Communication Complexity

Entry 46ff5a6b3292 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 11:36:48 UTC

Coxeter Group Action Index and Communication Complexity

Entry ID: 46ff5a6b3292 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 11:36:48 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Group Theory **Field B** (complexity object): Communication Complexity

Statement:

For every instance φ of communication complexity with binary inputs, the minimal number of generators of any Coxeter group action on the set of possible outcomes of φ is $O(\alpha(\log n))$, where α is the Ackermann function and n is the length of the input string.

Rationale (proposer's reasoning):

Coxeter groups provide a combinatorial framework that can model symmetries in discrete structures, potentially capturing hidden patterns in communication complexity problems. The minimal number of generators can serve as an invariant that reflects the structural complexity of the communication process, which might correlate with the communication complexity class.

Taxonomy category: combinatorial_group_theory_communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 06f4f6ae8b5a9257

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated communication complexity instances φ with input length n , the ratio of the computed minimal number of generators to $\alpha(\log n)$ is within $\pm 10\%$ for at least 80% of the seeds and has an aggregate mean ≤ 3 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 1 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Coxeter Group Action" AND "communication complexity" - "combinatorial group theory" AND communication complexity AND Coxeter - "Ackermann function" AND "Coxeter group action" AND communication

Top relevant hits considered: - [<http://arxiv.org/abs/2209.08813v2>] Cactus groups, twin groups, and right-angled Artin groups

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def ackermann(m, n):
    if m == 0:
        return n + 1
    elif n == 0:
        return ackermann(m - 1, 1)
    else:
        return ackermann(m - 1, ackermann(m, n - 1))

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_communication_instance(n):
        inputs = [''.join(random.choice('01') for _ in range(n)) for _ in range(2**n)]
        outcomes = [random.randint(0, 1) for _ in range(2**n)]
        return inputs, outcomes

    def compute_minimal_generators(inputs, outcomes):
        n = len(inputs[0])
        generators = set()
        for outcome in outcomes:
            generator = []
            for i in range(n):
                if outcome & (1 << i):
                    generator.append(i)
            generators.add(tuple(sorted(generator)))
        return len(generators)
```

```

def alpha(log_n):
    if log_n == 0:
        return 1
    elif log_n == 1:
        return 2
    else:
        a = 2
        for _ in range(1, log_n):
            a = 2 ** a
        return a

n_values = [5, 10, 15, 20, 30, 40]
total_metric_value = 0
instances_tested = 0
n_max = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    inputs, outcomes = generate_communication_instance(n)
    min_generators = compute_minimal_generators(inputs, outcomes)
    alpha_log_n = alpha(int(math.log2(n)))
    metric_value = Fraction(min_generators, alpha_log_n)

    total_metric_value += metric_value
    instances_tested += len(outcomes)
    n_max = max(n_max, n)

    if abs(metric_value - 1) > Fraction(10, 100):
        conjecture_holds = False
        counterexample = f"n={n}, min_generators={min_generators}, alpha_log_n={alpha_log_n}"

mean_metric_value = total_metric_value / instances_tested

return {
    "metric_name": "minimal_generators_over_alpha_log_n",
    "metric_value": float(mean_metric_value),
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

```

```

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=NA support_fraction={support_fraction}")
elif any(r["counterexample"] for r in results):
    first_failing_seed = next(r["seed"] for r in results if r["counterexample"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not meet the pre-registered support condition for the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	26552
2	preregistration	ollama_remote	glm4:latest	0	0	9273
3	novelty	ollama_remote	glm4:latest	0	0	11042
4	novelty	ollama_remote	glm4:latest	0	0	15337
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13233
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15257
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10760
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16912
9	judge	ollama_remote	glm4:latest	0	0	13423

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 131789 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/46ff5a6b3292.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/46ff5a6b3292.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/46ff5a6b3292.tar.gz` (if generated)